



Slide 1
CS 170
Java Programming 1
Objects and Methods
 Duration: 00:01:14
 Advance mode: Auto

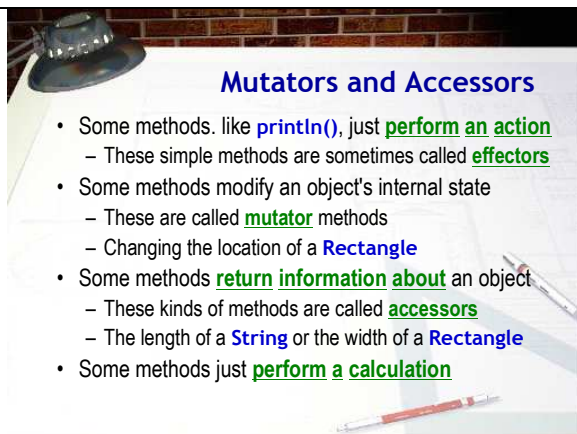
Notes:

Hi Folks. This is the CS 170, Java Programming 1 mini-lecture on Objects and Methods.

In the last lesson, you learned how to create customized objects by supplying specific values to the object's constructor in the form of actual parameters. Putting that new knowledge to work, you used BlueJ's "Use Class Library" facility to construct a Rectangle using the four-int-parameter version of the constructor. Once you created the object, you used the BlueJ's Inspect facility to make sure that the parameters you supplied were correctly stored inside each of the object's instance variables or fields.

Finally, at the end of the lecture, you instructed your object to change its location, by calling the setLocation() method, supplying the method with a new set of actual parameters. Again, when you inspected the object, you found that the x and y fields took on the new values.

That's where we want to pick up with this lecture. Make sure you have BlueJ open, and your Rectangle object (I'll use the name r1 for my Rectangle) open on the Object bench. Once you're ready to go, we can talk a little more about objects and methods.

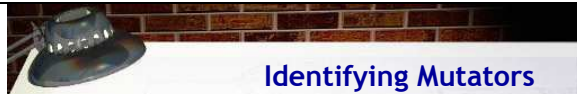


Slide 2
Mutators and Accessors
 Duration: 00:01:27
 Advance mode: Auto

Notes:

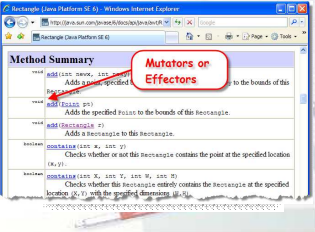
Objects have several different kinds of methods:

- Some methods, like `println()` for instance, carry out an action, but don't change the object that carries out the action. There's no official CS term for these kinds of methods, but I usually refer to them as effectors, a term used in robotics for an actuator.
- Some methods carry out an action, and make a change to the object that carries out the action. These are called mutator methods, because they change the object itself, just like a genetic mutation does. The Rectangle's `setLocation()` method is a mutator, because after you call it, both the x and y fields potentially have a new value. A mutator (potentially) changes the state of an object.
- Some methods return information about a particular object. These kinds of methods, called accessors, may tell you the number of characters in a String, or the width of a Rectangle. Since an object's data members are always private, accessors are the only way that the outside world can discover the state of a particular object.
- Finally, some methods just perform a calculation and return the calculation to the person who called the method. Most mathematical methods fall into this category. I'll call such methods calculators.



Identifying Mutators

- The documentation for effectors and for mutators start with the keyword **void** in the left-hand column
 - `System.out.println()` and `Rectangle's setLocation()` are **void** methods
- Exercise 1:** Locate another mutator method in the **Rectangle** documentation. Call the method and shoot 2 pics.



Slide 3

Identifying Mutators

Duration: 00:01:55
Advance mode: Auto

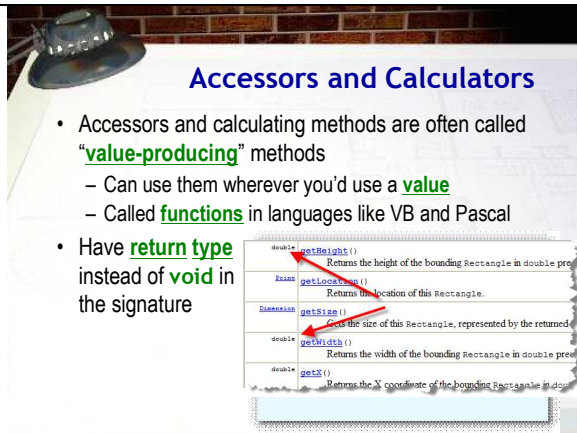
Notes:

So, how do you tell if a method is an effector, a mutator, an accessor or a calculator? Are they labeled like that? No, not exactly, but by carefully reading the documentation, you can usually tell what kind of method it is. We'll start by identifying effectors and mutators.

If you've closed the Documentation for the Rectangle class, open it again by choosing Use Library Class and clicking the Documentation button as shown here. When your browser opens up (at the Constructor Summary section), scroll down to table headed Method Summary, as shown here.

In the method summary, locate the left-hand column. Notice that the constructor summary had only a single column. This left hand column is where the method's return type goes. Since effectors and mutators don't return or produce any kind of value, they'll have the keyword **void** in this column. If you scroll down and locate the `setLocation()` method, you'll see that it is a void method. The `PrintStream's println()` method is also a void method.

OK, let's see if you understand this. In the last lesson, you called the `setLocation()` mutator method. Use the documentation to locate another Rectangle mutator method, and call the method using Code Pad and the Rectangle object on the object bench. For Exercise 1 of this lecture, snap me a picture of both your Code Pad window and the Object Inspector window, showing that the object's fields have been changed after calling the method. (Remember to create a new section for this lesson inside this week's IC document.)



Accessors and Calculators

- Accessors and calculating methods are often called "value-producing" methods
 - Can use them wherever you'd use a **value**
 - Called **functions** in languages like VB and Pascal
- Have **return type** instead of **void** in the signature

```

double getHeight()
    Returns the height of the bounding Rectangle in double precision.
int getX()
    Returns the location of this Rectangle.
int getWidth()
    Returns the size of this Rectangle, represented by the returned value.
double getY()
    Returns the width of the bounding Rectangle in double precision.
double getY()
    Returns the Y coordinate of the bounding Rectangle in double precision.
      
```

Slide 4

Accessors and Calculators

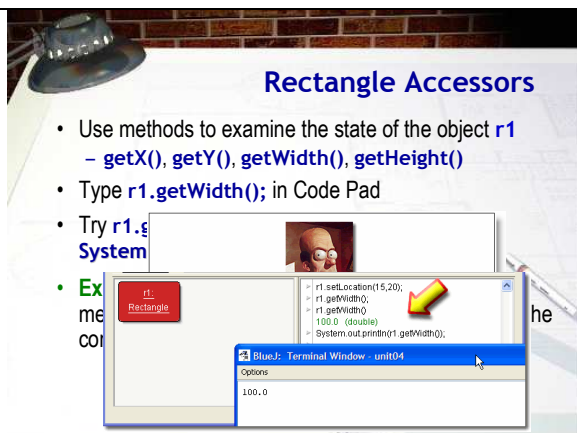
Duration: 00:01:07
Advance mode: Auto

Notes:

Both methods that provide information about an object, and methods that perform a calculation are often called "value-producing" methods, because you can use the method in any situation where you could use a value of the same type. In this picture here, the roll() method of the Dice class is a value-producing method, since it returns a value to the player.

In some other programming languages, such as Visual Basic, and Pascal, these kinds of methods are named functions, and I'll use that short-hand term to refer to value-producing methods, unless I'm specifically referring to an accessor method.

So, how do you recognize a function? Scroll on down through the methods, looking at the left-hand column once again. If the column does not contain the word void then it is a "function." The type you see listed here, int, double, String, and so on, is the kind of value that the method returns.



Rectangle Accessors

- Use methods to examine the state of the object **r1**
 - getX(), getY(), getWidth(), getHeight()**
- Type **r1.getWidth();** in Code Pad
- Try **r1.getHeight();** in Code Pad
- Example: **System.out.println(r1.getWidth());**

```

1: Rectangle
2: r1.setLocation(15,20);
3: r1.getWidth();
4: r1.getHeight();
5: System.out.println(r1.getWidth());
Options
100.0
      
```

Slide 5

Rectangle Accessors

Duration: 00:02:24
Advance mode: Auto

Notes:

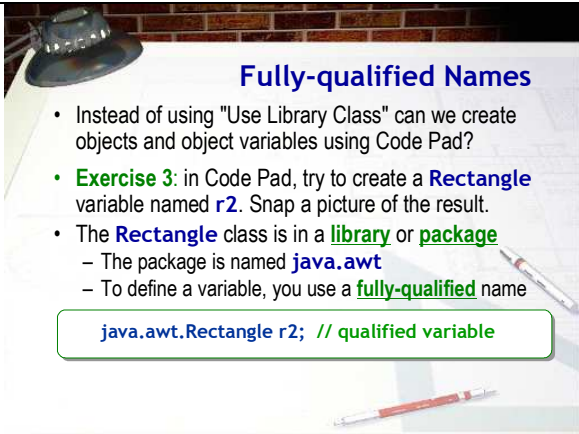
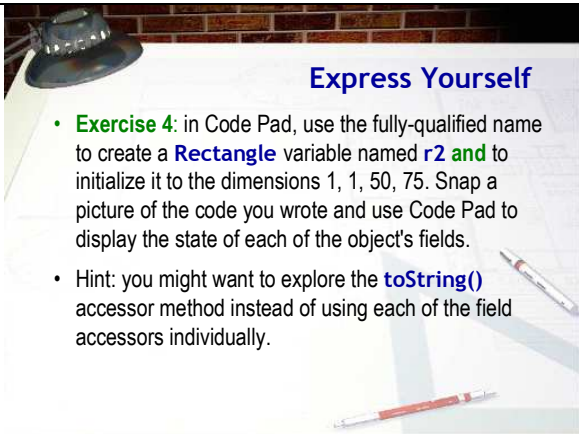
So, let's look at some accessors in the Rectangle class to examine the state of the Rectangle you've created, (instead of using BlueJ's Inspect Object dialog). The methods we'll look at will be getX() and getY() which will tell us about the location of the Rectangle, and getWidth() and getHeight() which will tell us about its size.

Let's start with the Rectangle width. Go ahead and type **r1.getWidth();** into Code Pad. Remember, of course, to use the name that you've used for your Rectangle object, if you used something other than **r1**. Do you get any output? No! Why is that? Remember, these are value-producing methods; they don't do something that you can see—instead, they give you back some information that you can use.

There are two ways you can see the result of a value-producing method in Code Pad. Let's look at both of them.

Type **r1.getWidth();** omitting the semi-colon into Code Pad. Without the semi-colon, you have an expression, not a statement, and Code Pad will attempt to evaluate the expression like this. Note that you can't do this with mutator or void methods, since they don't have any returned value. You can only use void methods as part of a statement.

The other way you can display the result of function is to pass the result you get back to another method that will display it, like **System.out.println()**. This might look kind of strange to you the first time you see it, but it's really very common. When you call a value-producing method, like **getWidth()**, inside the parameter list of another method, the first method is called and then the returned value is passed as the actual argument to the second method, in this case **System.out.println()**.

	OK, now it's your turn. Look up another accessor in the Rectangle class and then call it from Code Pad. Use both methods that I've shown here to display your output. Show me screenshots of both the Code Pad window and of the BlueJ console window showing your result.
 Fully-qualified Names - Instead of using "Use Library Class" can we create objects and object variables using Code Pad? Exercise 3: in Code Pad, try to create a Rectangle variable named r2. Snap a picture of the result. - The Rectangle class is in a library or package - The package is named java.awt - To define a variable, you use a fully-qualified name <pre>java.awt.Rectangle r2; // qualified variable</pre> Slide 6  Fully-qualified Names Duration: 00:01:40 Advance mode: Auto	Notes: So far in this lecture we've used the Use Class Library facility to create our object (placing it on the Object Bench), and then used Code Pad to manipulate the object. We can also use Code Pad to create Library-class objects and object variables as well, just like we did with the Circle object in the first lesson this week. This is a little closer to what you'll actually do when you write your programs. So, let's try that out with the Rectangle class. Try to use Code Pad to create a Rectangle variable (not an object) named r2. You'll probably get an error message. Whether you do or don't, for Exercise 3 shoot me a screenshot of the result. The reason you can't create a Rectangle variable, when you could create a Circle variable, is because the Rectangle class is located inside of a library or package. Each of the Java class library packages has a specific name. The package containing the Rectangle class, for instance is named java.awt. To create an object of a class inside a package, you have to use what's known as the fully-qualified name, like this one for the Rectangle class. Think of it like a formal name like "Joseph S. Student who lives at 123 Main Street in Irvine California". The fully qualified name let's the Java compiler and runtime (and BlueJ in our case) unambiguously locate the class you're interested in.
 Express Yourself Exercise 4: in Code Pad, use the fully-qualified name to create a Rectangle variable named r2 and to initialize it to the dimensions 1, 1, 50, 75. Snap a picture of the code you wrote and use Code Pad to display the state of each of the object's fields. - Hint: you might want to explore the toString() accessor method instead of using each of the field accessors individually. Slide 7  Express Yourself	Notes: OK, it's your turn again. Now that you know how to use the fully qualified name, create a Rectangle object in Code Pad along with an object variable named r2, and initialize the object to the dimensions 1, 1, 50, 75. Note that you'll have to use the fully qualified name for the constructor as well. Once you've created your variable, use its accessor methods to display the state of each of the object's fields, since we can't use the Inspect Object dialog for variables and objects that aren't located on the BlueJ Object Bench. Here's a hint that may make your life a little easier. You might want to explore the toString() accessor method instead of using each of the field accessors individually. Most objects have a toString() method which can be used as a quickie

Duration: 00:01:01
Advance mode: Auto

"inspect object" command, especially if a class doesn't provide an accessor method for a specific field.

Using import

- Why not use the full name (**java.awt.Rectangle**)?
 - It's a lot of work and it clutters up your code
- Instead, normally use the **import** statement

```
import java.awt.Rectangle;  
import java.awt.*;
```

- Second version allows access to all classes in package
- This must appear before the class definition

• **Exercise 5:** in Code Pad, use import and then make a **Rectangle** variable named **r3**. Snap a picture of the code you wrote and inspect the object.

Slide 8

Using import

Duration: 00:01:56
Advance mode: Auto

Notes:

You can write all of your programs using the fully-qualified names for every Java library class you need to use. Nobody does that, though, because it's a lot of extra typing, and, more importantly, it really clutters up your code, making it hard to read and understand.

Instead, you'll almost always use the import statement. If we wanted to refer to the Rectangle class using only the class name, we could use either one of these forms of the import statement. The first one selects a single specific class, in this case, the Rectangle class. The second version, using the `.*` wildcard, allows you to refer to any class in the `java.awt` package by using simply the class name. Some instructors, and your textbook, encourage you to only use the first version since it requires you to specifically identify which classes you want to use in your program. You may find that you have to add a number of import statements to your code. I don't have a problem with you using the second form of import if you like. Your code may compile slightly faster using the first form, but the size of your code will be identical in both cases.

The import statements have to appear before any class definitions. Generally, you'll add them as the very first line in your code, even before the JavaDoc comment that introduces each file.

OK, to finish up this mini-lecture, let's try using import in Code Pad. Import the `java.awt.Rectangle` class and then construct a third rectangle variable named `r3`. You can use any initial parameters you like. Snap me a picture of the code you wrote and use one or more of the accessors to display the state of the object.